

Asynchronous Communications Through Messaging

🌐 systemdesignschool.io/fundamentals/microservice-async-comm



Now that we have a comprehensive understanding of synchronous communication, let's take a look at asynchronous communications.

At its core, asynchronous communication relies on sharing data between components in a non-blocking way. This data exchange can take many forms, but messaging systems are among the most common.

What Is a Message?

A message is the fundamental unit of communication in asynchronous systems. It consists of:

- Header: Key-value pairs that describe metadata, such as the sender, timestamp, message type, or priority.
- Body: The actual data payload, which can be in text or binary format. The format of the body is not predefined (e.g., JSON or XML) and must be defined by the system's creators.

Here's a simple ASCII representation of a message:

```
+-----+
| Header |
| message_id: 12345, timestamp: 1678901234567 |
+-----+
| Body   |
| { "order_id": "A456", "status": "confirmed" } |
+-----+
```

In this example using RabbitMQ, the message header includes a unique `message_id` for tracking and a `timestamp` indicating when the message was created. The body contains an order ID and its current status, demonstrating a common use case in e-commerce systems to asynchronously notify inventory services about new orders.

How Asynchronous Messaging Works

Asynchronous messaging systems enable different styles of communication based on the use case:

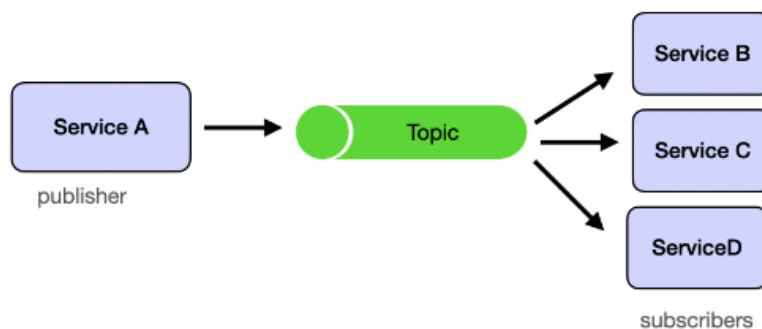
One-Way Notification

A sender transmits a message to a recipient without expecting a response. This is useful for fire-and-forget scenarios, such as logging or sending alerts.



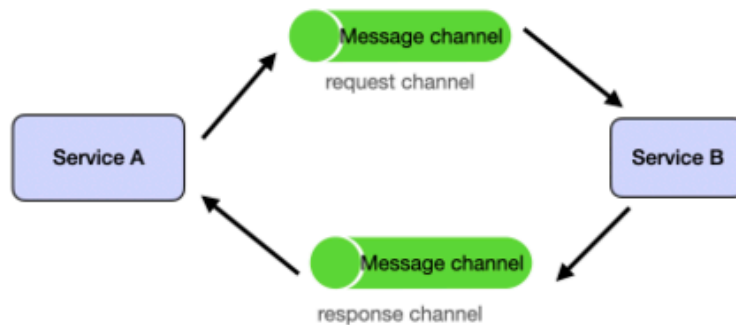
One-Way Pub/Sub

A sender publishes messages to a topic, and multiple subscribers listen for updates. This is common in event-driven architectures where many components react to the same event.



Two-Way Async Request-Response Style

The sender sends a request message and later receives a response message. This requires two channels. This style is not as widely used as the first two.



Message Broker

You may have noticed the green “message channel” in the figures. This is typically implemented using a message broker, an intermediary through which all messages flow. A sender writes the message to the message broker, and the message broker delivers it to the receiver. It ensures reliable delivery, manages message queues, and supports various messaging patterns like point-to-point and publish-subscribe.

Popular message brokers include RabbitMQ, Apache Kafka, and Amazon SQS. Each message broker implements the message channel concept in a different way.

Message Broker	Channel Name Type
RabbitMQ	Queues, Topics
Apache Kafka	Topics (Partitions)
Amazon SQS	Queues
Google Pub/Sub	Topics

Pros and Cons of Asynchronous Communication

Pros

- **Loose Coupling:** Components don’t need to know about each other’s existence.
- **Buffering:** Messages can be stored temporarily, allowing receivers to process them at their own pace.
- **No Service Discovery Needed:** Senders only interact with the message queue or topic, avoiding the complexity of finding specific receivers.

Cons

- **Complexity:** Managing message brokers and ensuring reliability can add operational overhead.
- **Latency:** Asynchronous systems might introduce delays, especially when messages accumulate in queues.

- Error Handling: Debugging can become more challenging due to the decoupled nature of the system.